

AI & Prompt Engineering

From Data to Decisions - Thinking in Systems



Roadmap

1

System

2

Data

3

Communication

4

Projects

5

Assistants

6

Agents

Foundations

What is AI, really?

Autocomplete

- Predictive text on your phone - except this is an LLM, a Large Language Model trained on an almost unimaginable amount of human writing, so its guesses are fluent and often startlingly good. That's what makes it generative rather than just predictive, like a spam filter sorting mail into categories: it produces new content, not a fixed label. Still fundamentally guessing the next word, not looking up a fact.

Neural network

- Loosely modeled on neurons in a brain - layers of simple units that combine to recognize complex patterns in language.

Chatbots

- That same prediction engine, wrapped in a conversational interface so you can talk to it in plain language instead of code.

► Want a plain-English explainer? [Watch: what AI is, in plain English \(IBM Technology, ~10 min\).](#)

Foundations

Not all AI is the same

Different jobs

- AI isn't one thing. AlphaFold predicts protein structures, image models generate pictures, Claude works with language - each built and trained for a specific kind of task.

Narrow is strong

- A model brilliant at one job is often useless at a completely different one. Power comes from focus, not from being generic.

Why LLMs here

- This course is about using Claude, an LLM, well - a deliberate choice, since it's the tool you'll actually have access to. Other AI types exist and matter; they're just not what we teach here.

Your Claude Account

Log in at claude.ai with your CUNEF academic account.

Take two minutes now to confirm you can get in - from Part 2 onwards, every part has a hands-on component.

► New to this in Claude? [Watch](#) · [Read](#).

1

From Chat to System

1 | From Chat to System

Why does it behave the way you've probably noticed?

Hallucinates

- Like a brilliant, overconfident dinner guest: fluent about almost anything, but hasn't fact-checked recently. It completes the most probable pattern with total confidence - not lying, just guessing well.

Knowledge cutoff

- Training data stops at some point - anything after that, it doesn't know unless told or it searches.

Limited memory

- A context window limits how much conversation it can hold in mind at once.

1 | From Chat to System

From each habit, a rule

Verify facts

- Numbers, names, dates, citations - check them, the same way you'd check a claim from any single source.

Stay current

- For anything current or changing, hand it the information yourself or ask it to search.

Remind it

- In a long conversation, repeat what matters or start fresh. Part 4 covers a much better fix: Projects.

From doer to director

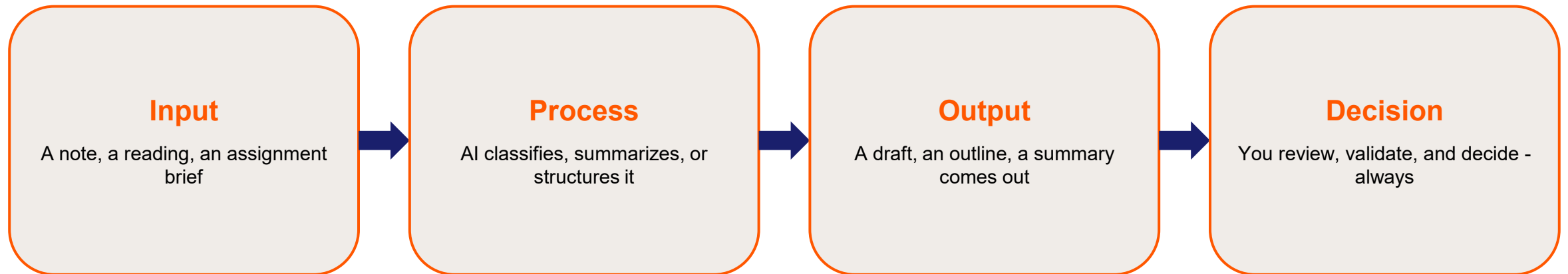
Executor

- Does each task manually, over and over
- Retypes the same instructions each day
- Result depends on repeating itself well
- Loses hours each week to recurring work

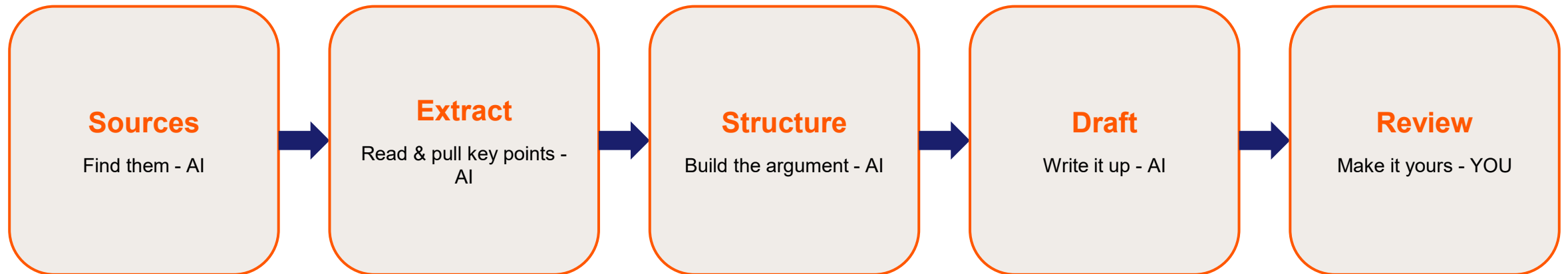
Director

- Designs the process once, lets it run
- Leaves rules saved - no repeated instructions
- Result scales without repeated intervention
- Dedicates time to deciding, not executing

AI as a system



The secret: break it down



1 | From Chat to System

The limit is you, not the tool

So we're going to learn to minimize that human bottleneck.

Clarity = speed

- If what you give is messy, you'll get messy back; a simple, clear brief goes a long way.

You add value

- AI gives you a first draft; you bring the tone, judgment, and nuance it doesn't have.

Risk = control

- A summary just for you tolerates small errors; an exam date or a citation you submit doesn't.

AI is a fast, brilliant collaborator - but it has no stake in the outcome and no context of its own. The judgment and the context are yours.

2

Data and Grounding

You already know it hallucinates

Unanchored

- Invents with total confidence
- Argues beautifully while fabricating the evidence
- Sounds convincing about something false

Anchored

- Works only with the sources you give it
- Gives you the receipt - exact citations for every claim
- Can't invent what it's not allowed to invent from

What is grounding?

Grounding: forcing AI to work exclusively with the sources you give it. It doesn't just give you an answer - it gives you the receipt.

This is the ground floor of everything else in this course. Projects, Assistants, and Agents are all built on top of it.

And when an answer disappoints, the fix is almost always better sources - not a fancier model.

2 | Data and Grounding

The method

Attach the source

- Upload or paste the real document into the chat - a syllabus, your notes, whatever you need it to work from.

Ask what only it can answer

- Not a generic question - something only your document could answer, so you know it used your source and not its training.

Check it traces back

- A grounded answer points back to your source. If it can't, or it sounds generic, that's your signal it drifted off your material.

Let's build it together

1. Open a new chat at claude.ai
2. Upload the course syllabus (PDF)
3. Ask it: “What's the minimum grade needed on the final exam to pass this course?”
4. Compare: ask the exact same question in a different chat, with nothing uploaded

Your turn

Pick a different course you're taking right now. Do the same thing: upload your real notes or syllabus into a new chat, and ask one specific question only that material can answer.

2 | Data and Grounding

Real cases for grounding

Study syllabus

- A dense 500-page reading list becomes outlines, mind maps, or flashcards.

Quiz yourself

- Generate test-style questions from your own notes and find your actual gaps.

Interview prep

- Cross your CV with a job posting and spot your weak points before the interview.

The competitive advantage is never the tool. It's the system you build. You'll all have access to the same AI - the difference is who anchors it to better knowledge.

3

Communication

3 | Communication

Three blockers to generic answers

Not psychic

- You write six words and expect it to guess your situation. Fixed by giving it structure and context.

Can't translate

- You know what you want but not how to explain it. Fixed by letting AI interview you, or starting from an example.

Not more typing

- You exhaust yourself copying huge blocks into every chat. Fixed by systematizing your best instructions.

3 | Communication

ICC: drafting from scratch

Instructions

- The exact task and action you want it to execute.

Context

- Your role, your objective, your background, and who the output is for. Where most failures happen, and what personalizes the result most.

Constraints

- The rules: style, tone, length, format. Plus an example to imitate, if you have one.

ICC in action

A deadline-extension email

- Instructions: draft a short, polite email requesting a deadline extension
- Context: first-year student, missed 3 days due to illness, doctor's note available
- Constraints: under 100 words, formal tone, no excuses beyond the illness

A weekly study plan

- Instructions: write me a simple weekly study schedule
- Context: 5 courses this semester, 2 free evening hours most weekdays, one weak subject: statistics
- Constraints: one page, extra time blocked for statistics, keep weekends mostly free

3 | Communication

AIM: directing an expert

Actor

- Tell it who it is - assign it an expert role.

Input

- Give it the material - upload your documents and data.

Mission

- The ultra-specific, measurable goal you're pursuing.

AIM in action

Improve my CV

- Actor: You're the most sought-after CV editor in the world
- Input: my CV + the internship posting + company notes
- Mission: 5 concrete improvements to raise my interview chances

Plan a trip

- Actor: You're a savvy budget-travel planner who's done this a hundred times
- Input: my dates, my budget, a photo of my current itinerary notes
- Mission: a 3-day plan under my exact budget, with 2 backup options

When to use each

ICC

- For drafting from scratch
- Email, summary, or idea with no prior material
- Focus: explain the task well

AIM

- For directing an expert with material
- Analyzing, reviewing, or optimizing something that already exists
- Focus: the role and the mission

Let AI interview you

The magic phrase: “Ask me whatever questions you need to do a better job.”

Example: you ask “Create a study plan for this semester” + the phrase. AI pauses and asks: which subjects? when are your exams? how many hours a week do you have? Only then does it deliver a plan built for you.

3 | Communication

Reverse prompting

Can't phrase the ICC or AIM prompt yourself? Hand over an example and let AI reverse-engineer it for you.

A document

- Give it a report or table you like; it generates yours in that same format.

Your style

- Pass it your best texts; it clones your tone as a reusable skill.

Independence

- Stop asking for prompts on social media - the reference plus AI writes the prompt for you.

3 | Communication

More ways to ask well

Not new frameworks - precision you drop into the Instructions or Constraints of any ICC or AIM prompt.

Few-shot

- Show it 2-3 examples of exactly what you want instead of describing it - it copies the pattern.

Step-by-step

- Ask it to think through the problem in steps before answering - catches errors a rushed answer would miss.

Output shape

- Ask for the exact shape you need - a table, a specific template - not just 'content.'

3 | Communication

Try it: a naive prompt

1. Upload this module's slides into a new chat.
2. Ask the simplest thing: “Summarize this.”
3. Then, in the same chat: “Now just the part on ICC and AIM.”

It answers the follow-up without you re-explaining - the chat remembers what's in it. Close that chat, though, and the memory is gone. (Part 4 fixes that.)

3 | Communication

Fix it with the Prompt Doctor

“Summarize this” is vague - no role, no audience, no shape. Before you settle for the generic answer, run your prompt through the Prompt Doctor:

[Open the Prompt Doctor →](#)

It checks the six things a strong prompt needs, tells you whether ICC or AIM fits, and rebuilds the prompt with you - one field at a time.

The Prompt Doctor itself is an Artifact - a small interactive tool built with Claude. You'll learn to make your own in Part 4.

Raw vs. improved

Raw prompt

- “Summarize this”
- A generic, whole-deck summary
- Correct - but not useful to anyone in particular

Improved (ICC / AIM)

- A role, an audience, and a shape added
- “...for a friend who missed class, in 4 plain bullets”
- Scoped, ordered, actually usable

**Stop trying hard to speak
AI's language. Make AI work
to understand you.**

4

Projects

You repaste this every time

Every new chat starts blank. You explain again who you are, your tone, and your rules... over and over.

Starting a chat from zero each time is like hiring a new employee every day and spending the first hour bringing them up to speed. The cost isn't visible, but it's there: wasted time, and an AI that never reaches its full potential.

Chat vs. Project

Regular chat

- Starts blank every time
- You repeat your context and instructions
- Closing it, AI forgets everything

A Project

- Your style and rules already “baked in”
- Your documents live inside, anchored
- Persistent knowledge - you stop re-explaining

4 | Projects

A thematic folder with memory

Instructions

- Its role, your tone, and the format rules it should always follow.

Sources

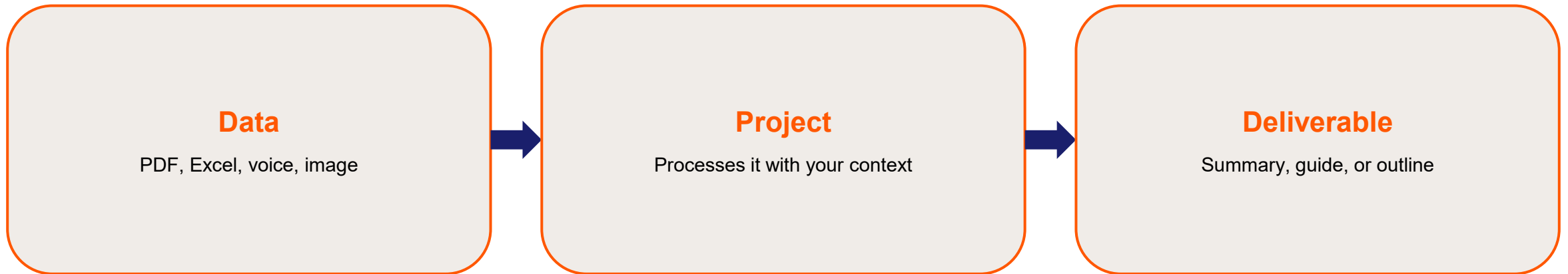
- Notes, PDFs, guides, images - the base knowledge it draws from.

Isolated memory

- One project's stuff doesn't contaminate another's. Shareable with your group to work from the same base.

► New to this in Claude? [Watch](#) · [Read](#).

The flow



4 | Projects

What's hardest to understand

Not assistant

- A Project handles several tasks on one topic. One repetitive task is a different tool - that's Part 5 (Assistants).

Not a drive

- A folder remembers where files are; a Project remembers how you work. Configure rules and tone, not just upload stuff.

Hidden cost

- Starting from zero each time seems free. It's not - it's the 'employee' who loses their memory daily.

4 | Projects

Meet Artifacts

Not just text

- A live, testable canvas that sits next to the chat - a real quiz, a flashcard deck, a diagram, not just words.

Built from data

- Ask for one from inside your Project, and it draws on the sources you've already uploaded.

Yours to keep

- Download it, keep it in the Project, or come back to it anytime.

► New to this in Claude? [Watch](#) · [Read](#).

Let's build one together

1. Actor: “You're a quiz-maker for university courses”
2. Input: this course's syllabus
3. Mission: “a 5-question self-scoring practice quiz about this course's assessment structure”
4. Constraint: ask for it as an Artifact, not just a text answer

Your turn

Create a Project for a course you're actually taking. Upload your real notes. Then ask for a self-scoring quiz or flashcard set from those notes - AIM again. Keep it: this is now a real study tool for that course, not just today's exercise.

A folder remembers where your files are. A Project remembers who you are, what matters to you, and how you think. Projects don't organize files - they organize behavior.

5

Assistants

Do not Repeat Yourself (DRY)

When a task is always the same, stop retyping long instructions - package it once and reuse it.

- The summary of every reading.
- Correcting every draft.
- Answering the same recurring question.

Project vs. Skill

Project

- Several distinct tasks on one topic
- Example: organize an entire course - notes, practices, doubts, calendar
- You still steer each task

Skill

- The same identical action, repeated
- Example: generate a summary of every reading, the same way each time
- It runs the same method on its own, every time

5 | Assistants

A preconfigured specialist

Fixed role

- Give it permanent behavior, not new instructions every time.

Reference docs

- Its anchored sources - it works with your information, not generic knowledge.

Default style

- Respects your output format, starts with the structure you've already decided.

5 | Assistants

You've already used one

The Prompt Doctor you used in Part 3 was really three small Skills working together.

Evaluate

- Checks a prompt against the six things a strong one needs, and asks what's missing.

Rewrite to ICC

- Turns a from-scratch request into a clean Instructions-Context-Constraints prompt.

Rewrite to AIM

- Turns a request that has material into a clean Actor-Input-Mission prompt.

5 | Assistants

Give it structure

There's no visual formatting in an instructions box. A flat wall of text is hard for it to parse - give the instructions real hierarchy instead:

```
# Role
```

```
You are a tutor for this course.
```

```
## Rules
```

- Explain step by step
- Use simple examples

```
## Output format
```

```
Summary + 3 review questions
```

► New to this in Claude? [Watch](#) · [Read](#).

Prompt vs. Skill

A prompt

- Instructions for one specific dish
- Knows what to do this time
- Doesn't know your processes or templates - a brilliant intern

A skill

- The steps you followed, saved for reuse
- Activates itself in the background
- Runs with your style, no giant prompt repeated

Let's build one together

1. Role: a Socratic tutor - it never just hands you the answer
2. Rules: ask one guiding question at a time; find where my reasoning went wrong; bigger hints only if I'm truly stuck
3. Output format: short questions, not answer-dumps; stop once I can restate the idea myself
4. Save it as a Skill - now it tutors you on anything, and Part 6's agent will use it

Your turn

Think about the course you built a Project for in Part 4. What do you do the exact same way, over and over, for it? Build a Skill for it - a different tool, same course, ready forever.

Stuck for an idea? Build a note-summarizer: “turn any lecture notes into the same 5-bullet summary + 3 review questions, every time.”

A bit of security judgment

What you expose

- Don't show sensitive data to tools that read your screen. On delicate topics, validate through an official channel.

Who gets access

- Sharing a Project? Isolate its memory and choose permissions - just use it, or also edit it.

What you install

- Third-party skills are the biggest risk - they can act on your behalf. Install only well-recognized ones.

The most important thing about an assistant isn't the model or the tool. It's the instructions - they define its behavior.

6

Agents

As a tool vs. as an agent

As a tool

- You do the work, leaning on it
- You ask for a text, an idea, a summary - step by step, click by click
- It waits for you between every step

As an agent

- You set the goal, it decides the steps
- It plans the sequence and works through it on its own
- It reports back with the result, not a request for the next click

► What's an agent, exactly? [Watch: what AI agents are \(IBM Technology\)](#).

Your new role

Background work

- It works in the background, connected to your tools - it pulls what it needs without you lifting a finger for each step.

While you sleep

- A beginner opens AI only when they need something; someone experienced already has agents solving tasks by the time they wake up.

Now you direct

- Your job stops being typing the perfect prompt for every step - it becomes orchestrating the system and validating the result.

6 | Agents

What MCP does

MCP = Model Context Protocol

Reads

- Pulls in what it needs from the tools you already use - notes, a calendar, files - not just what's typed in the chat.

Bridges

- One shared connection standard, instead of you manually copy-pasting between every different app.

Also acts

- It can write back too - send something, update something - which is exactly the part worth watching closely.

► New to this in Claude? [Watch](#) · [Read](#).

Routines: tasks that trigger themselves

By schedule

- Daily, weekly, or monthly. Runs at the fixed time, no matter what.

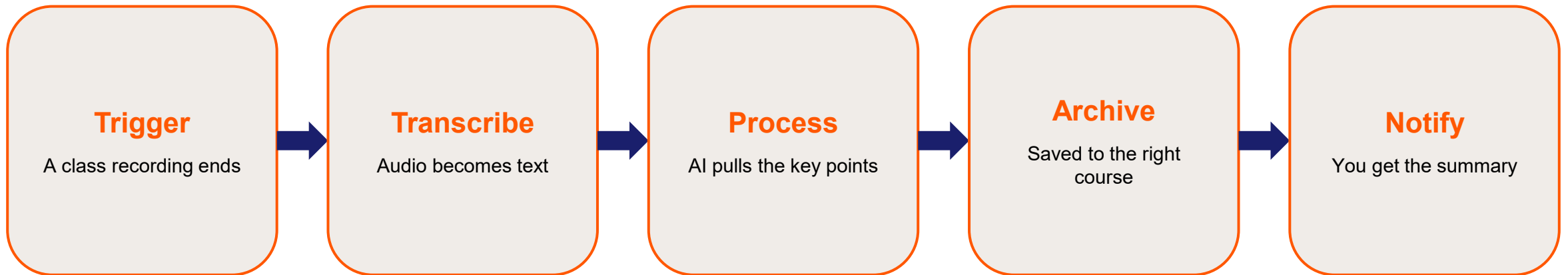
By event

- The most powerful kind: fires the instant something happens - a message arrives, a class ends.

In the cloud

- Runs outside your own device - your laptop doesn't even need to be on.

One step feeds the next



Who validates?

Low risk

- Sorting files, drafts, a summary just for you
- A mistake is cheap - it self-corrects or costs you nothing
- Let the agent run and check the result after

High risk

- An exam date, a citation, a message sent to someone else
- The cost of a mistake is real and can't be undone
- Validate before it acts, not after

A bit of responsible judgment

Verify

- Whatever an agent hands you, you're still the one who checks it before it counts for something.

Disclose

- If you're asked whether and how you used AI on something, answer honestly - institutions tend to have AI-use policies, and you must comply.

Keep control

- Delegating the work isn't delegating the decision. The decision is always still yours.

6 | Agents

Let's build one together

1. Point it at the Project you built in Part 4 (your course + notes).
2. Goal: “Quiz me one question at a time on this material.”
3. “When I'm wrong, use my Socratic-tutor Skill - don't just give the answer.”
4. “If something isn't in my notes, search the web and tie it back.”

It decides which of those to do, step to step - Project, your Skill, or the web. Choosing on its own is what makes it an agent, not a tool.

► New to this in Claude? [Enable and use web search in Claude \(official\)](#).

Your turn

Point your own Study Buddy at your Part 4 Project. Let it quiz you, tutor you with your Skill when you slip, and fill gaps from the web. Keep it - real revision help, on tap, for a course you're actually taking.

Skill vs. Agent

A Skill

- A saved procedure for one repeatable job
- Reactive - you invoke it; it runs its method
- Technically: a file - SKILL.md (name, description, instructions)
- Ours: the Socratic tutor you built in Part 5

An Agent

- A goal-driven operator, not one fixed task
- Autonomous - it decides the steps, uses tools on its own
- Technically: not a file - a mode (a goal + tools + latitude); it can use Skills
- Ours: the Study Buddy - which runs your Socratic Skill

► See our two, side by side: [the Socratic Skill](#) · [the Study Buddy spec](#).

Discuss

Where's the line between an agent that helps and one you shouldn't trust unsupervised? Pick one to argue with the person next to you:

- Would you let an agent send an email to your professor on your behalf? What would have to be true first?
- Where's the line between “this saves me time” and “I've stopped understanding my own work”?
- If an agent hands you a confident answer with a made-up source, who's responsible - and who would even catch it?

Recap

1

System

2

Data

3

Communication

4

Projects

5

Assistants

6

Agents

Look at what you built

A Project holding your course. An Artifact to study from. A Skill that tutors you. And an agent that runs all three on its own. Four real layers, each built on the one before - and every one of them is yours.

► Keep going - the cheat sheet, the example Skills, and the Prompt Doctor all live here: [the AI module toolkit on GitHub](#).

**Humans for judgment.
Agents for execution.
Delegate the work - never the
decision. That's the one idea
this entire course has been
building toward.**



Campus Almansa | Calle Almansa 101 | 28040 Madrid

cunef.edu